

Дополнительная лекция по Си № 7

7 мая 2025 г.

Коновалов А.В.

Раздельная компиляция (практика)

Написали библиотеку для работы с двумерными векторами. Файл vec2d.h:

```
#ifndef VEC2D_H_
#define VEC2D_H_

struct Vec2d {
    double x, y;
};

struct Vec2d vec2d_add(struct Vec2d a, struct Vec2d b);
double vec2d_prod(struct Vec2d a, struct Vec2d b);
double vec2d_length(struct Vec2d v);
void vec2d_print(struct Vec2d v);

#endif /* VEC2D_H_ */
```

Файл vec2d.c:

```
#include <math.h>
#include <stdio.h>
#include "vec2d.h"

struct Vec2d vec2d_add(struct Vec2d a, struct Vec2d b)
{
    return (struct Vec2d) {
        .x = a.x + b.x,
        .y = a.y + b.y,
    };
}

double vec2d_prod(struct Vec2d a, struct Vec2d b)
{
    return a.x * b.x + a.y * b.y;
}
```

```

}

double vec2d_length(struct Vec2d v)
{
    return sqrt(vec2d_prod(v, v));
}

void vec2d_print(struct Vec2d v)
{
    printf("%.1f,%.1f", v.x, v.y);
}

```

Файл main.c, содержащий код, который использует эту библиотеку:

```

#include <stdio.h>
#include "vec2d.h"

int main()
{
    struct Vec2d a = { 1, 2 };
    struct Vec2d b = { 2, 2 };
    struct Vec2d ab = vec2d_add(a, b);
    double ab_len = vec2d_length(ab);
    printf("a = ");
    vec2d_print(a);
    printf("\nb = ");
    vec2d_print(b);
    printf("\nab = ");
    vec2d_print(ab);
    printf(", length = %.1f", ab_len);
    return 0;
}

```

Полная компиляция программы:

```
gcc main.c vec2d.c
```

При выполнении этой команды компилятор создаст для каждого исходного файла (единицы трансляции) объектный файл во временной папке, затем эти объектные файлы скомпирует в исполнимый файл a.exe или a.out в зависимости от ОС.

Раздельная компиляция осуществляется командами:

```

gcc -c main.c
gcc -c vec2d.c
gcc main.o vec2d.o

```

Опция -c предписывает скомпилировать файл без компоновки, в результате для исходного файла будет создан объектный файл с тем же именем, но расширением

.o. Последняя команда, получив на входе не исходники, а объектники, выполнит их компоновку.

Преимуществом раздельной компиляции является экономия времени: изменив один исходный файл (main.c или vec2d.c), достаточно перекомпилировать только один объектник, а затем выполнить компоновку. Перекомпилировать неизменённые файлы не требуется. При изменении заголовочного файла (vec2d.h в нашем примере) перекомпилировать необходимо все файлы, которые прямо или косвенно его включают.

Однако, как несложно догадаться, осуществлять раздельную компиляцию вручную трудоёмко. Для автоматизации этого процесса уже давно придумали соответствующие инструменты. Классическим примером является утилита make, которая, согласно сценарию в файле Makefile, выполняет сборку проекта.

У утилиты make есть масса разновидностей (каждый разработчик компилятора часто предоставляет свой вариант, например, вместе с Visual C++ устанавливается nmake), мы будем рассматривать GNU Make, используемую по умолчанию в GNU+Linux.

Утилита make универсальная, не заточена только на раздельную компиляцию программ на Си, она может автоматизировать сборку любых проектов со сложными зависимостями.

Синтаксис скрипта утилиты такой:

```
цель: исходники...  
      команда сборки...
```

Для каждой цели указывается, какие файлы для её сборки должны существовать, и какой командой (или командами) её можно получить из исходных файлов. Целей может быть несколько. По умолчанию собирается первая цель в файле.

Строки с командами сборки должны **обязательно** начинаться на **табуляцию**, а не пробелы!

Имя скрипта по умолчанию — Makefile (на unix-подобных системах, где регистр букв важен, имя должно начинаться с большой буквы).

Для нашего проекта можно написать такой make-файл:

```
vec2d-example.exe: main.o vec2d.o  
    gcc -o vec2d-example.exe main.o vec2d.o  
  
main.o: main.c vec2d.h  
    gcc -c main.c  
  
vec2d.o: vec2d.c vec2d.h  
    gcc -c vec2d.c
```

Программа make строит граф зависимостей между файлами. Файл пересоздаётся (выполняются команды сборки):

- либо когда он отсутствует (нужно выполнить соответствующие команды, чтобы файл создался),
- либо когда его дата последней модификации раньше, чем дата последней модификации исходных файлов.

Те файлы, которые существуют и созданы позже своих исходников (они называются пререквизиты), не пересобираются.

При использовании MinGW (портированного GCC на Windows) команда make называется mingw32-make.

Результат полной пересборки — выполнились все команды сборки (команда make печатает выполняемые команды):

```
D:\Си>mingw32-make
gcc -c main.c
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Вызовем сборку ещё раз:

```
D:\Си>mingw32-make
mingw32-make: 'vec2d-example.exe' is up to date.
```

Команда сообщила, что делать ей нечего.

Внесём небольшое изменение в файл main.c (например, добавим пустую строку) и снова выполним сборку:

```
D:\Си>mingw32-make
gcc -c main.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Перекомпилировался main.o и зависящий от него vec2d-example.exe.

Если удалим vec2d.o, то увидим похожий результат:

```
D:\Си>erase vec2d.o
```

```
D:\Си>mingw32-make
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Если изменим заголовочный файл vec2d.h, то перекомпилируется всё:

```
D:\Си>mingw32-make
gcc -c main.c
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Как упростить написание make-файлов? Можно использовать переменные и подстановочные знаки.

В командах можно использовать следующие переменные:

- `$@` — имя цели,
- `$+` — имена всех пререквизитов,
- `$<` — имя первого пререквизита.

Перепишем make-файл с использованием этих обозначений:

```
vec2d-example.exe: main.o vec2d.o
    gcc -o $@ $+
```

```
main.o: main.c vec2d.h
    gcc -c $<
```

```
vec2d.o: vec2d.c vec2d.h
    gcc -c $<
```

Убедимся, что всё работает:

```
D:\Си>erase *.o *.exe
```

```
D:\Си>mingw32-make
gcc -c main.c
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Мы видим, что сборка объектников выполняется одной и той же командой `gcc -c $<`. Для их сборки можно использовать шаблонное правило:

```
vec2d-example.exe: main.o vec2d.o
    gcc -o $@ $+
```

```
%.o: %.c vec2d.h
    gcc -c $<
```

Знак процента заменяет общую часть имени цели и пререквизита.

Команда `make` по умолчанию собирает первую цель, но ей можно указать сборку и произвольной цели:

```
D:\Си>erase *.o *.exe
```

```
D:\Си>mingw32-make main.o
gcc -c main.c
```

```
D:\Си>mingw32-make
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Вторая команда собирала один объектник. Третья команда показывает, что один объектник создан, его пересобирать не надо.

Синтаксис make-файла у GNU Make допускает указание ложных целей —

целей, которые будут собираться всегда, независимо от наличия одноимённого файла. Ложные цели описываются как пререквизиты у цели `.PHONY`, команда не указывается:

```
.PHONY: target1 target2
```

Как правило, ложными целями являются цели `all` (она обычно первая цель в файле) и `clean` (выполняет чистку от промежуточных и целевых файлов).

Добавим ложные цели в `make`-файл:

```
.PHONY: all clean
```

```
all: vec2d-example.exe
```

```
vec2d-example.exe: main.o vec2d.o
    gcc -o $@ $+
```

```
%.o: %.c vec2d.h
    gcc -c $<
```

```
clean:
    erase main.o vec2d.o vec2d-example.exe
```

Протестируем:

```
D:\Си>mingw32-make
mingw32-make: Nothing to be done for 'all'.
```

```
D:\Си>mingw32-make clean
erase main.o vec2d.o vec2d-example.exe
```

```
D:\Си>mingw32-make
gcc -c main.c
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

Первая команда сообщила, что нечего делать для сборки цели `all`, вторая выполнила очистку от генерируемых файлов, третья — показала, что после очистки потребовалась полная пересборка.

Если мы строчку с `.PHONY` закомментируем, то, создав файлы `all` и `clean` (без расширений), мы «обманем» команду `make`:

```
D:\Си>copy nul clean
Скопировано файлов:          1.
```

```
D:\Си>copy nul all
Скопировано файлов:          1.
```

```
D:\Си>mingw32-make clean
mingw32-make: 'clean' is up to date.
```

```
D:\Си>mingw32-make
mingw32-make: Nothing to be done for 'all'.
```

```
D:\Си>erase *.exe *.o
```

```
D:\Си>mingw32-make
gcc -c main.c
gcc -c vec2d.c
gcc -o vec2d-example.exe main.o vec2d.o
```

```
D:\Си>mingw32-make all
mingw32-make: Nothing to be done for 'all'.
```

Не, с целью all обмануть, всё же, не удалось, но с clean удалось.

Раскомментируем с .PHONY, цель clean заработает:

```
D:\Си>mingw32-make clean
erase main.o vec2d.o vec2d-example.exe
```

Цель all, как правило, нужна, если нужно создать не один файл, а несколько, например, несколько программ, или программу и документацию и т.д.

```
all: program.exe documentation.pdf
```

Также .PHONY-целями часто являются цели distrib и install. Цель distrib создаёт дистрибутив (архив или пакет установки .msi, .deb, .rpm), цель install копирует собранные файлы в каталоги ОС, как правило, требует рута.